

WAT4

Web Application Testing

Load Testing of Web Applications

P. Studener

Introduction & Motivation

Why Load Testing?

- Performance is a quality attribute (ISO 25010 – Performance Efficiency)
- Slow systems causes:
 - Revenue loss
 - SLA violations
 - Poor user experience
 - Reputation damage
- “It works on my machine” $\neq$ “It works under load”

Functional vs. Non-Functional Testing

Functional Testing	Non-Functional Testing
Does it work?	How well does it work?
Correct output	Performance, scalability
Usually deterministic	Often statistical

Load testing belongs to non-functional testing!

Types of Load Testing

Load Testing	Stress Testing	Spike Testing	Soak Testing
Expected traffic	Beyond capacity	Sudden traffic increase	Long-duration test
Validate SLOs	Find breaking point	Example: Ticket sales	Detect memory leaks
Realistic scenarios	Extreme scenarios		Detect resource exhaustion

Key Performance Metrics

Core Metrics – Per Service

- Response Time (average, median, percentiles p90/p95/p99)
- Throughput (requests/sec)
- Concurrent Users
- Error Rate
- CPU / Memory Usage
- Network Latency
- (Backpressure)

Why Percentiles Matters?

- Average → 200 ms
 - No issue
- p95 → 2.5 s
 - issue
- Important percentiles:
 - p50 (median)
 - p90
 - p95
 - p99

Key Performance Metrics

- What it means:
 - 95 % of all requests completed faster than this value, while 5 % were slower.
-
- Example:
 - If your API's p95 latency is 450 ms, it means 95 % of your users experienced a response time of 450 ms or less, while 5 % waited longer.

Workload Modeling

Why it matters?

- How many users?
- What user journeys?
- Think time?
- Read vs write ratio?
- Traffic distribution?

Load Profiles

- Constant Load
- Ramp-Up / Ramp-Down
- Spike Load

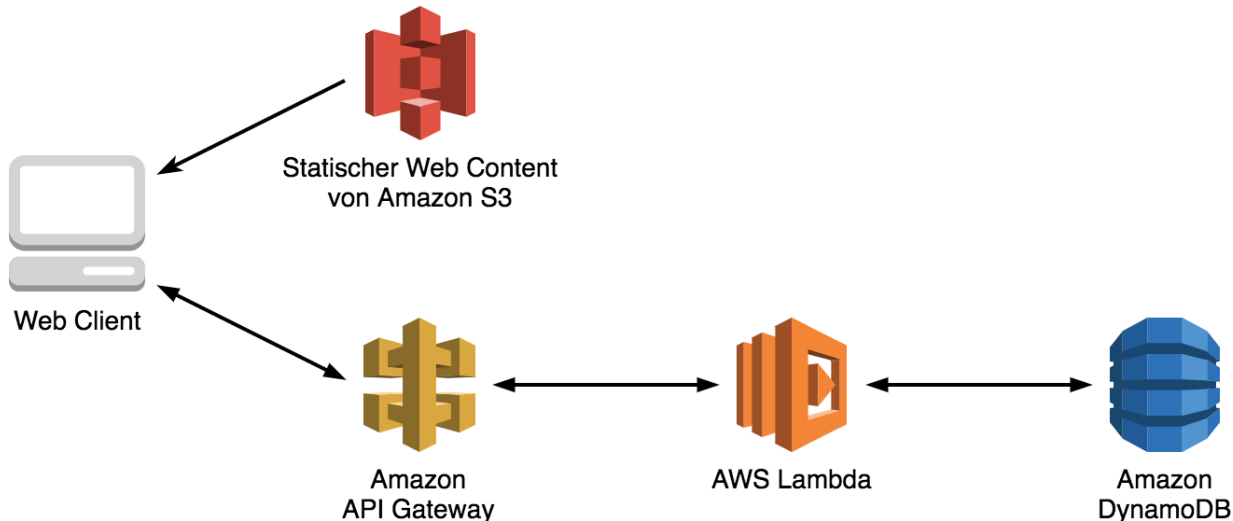
Think Time

- Think Time = Time between user actions
- Without think time:
 - Unrealistic load
 - Artificial bottlenecks
 - Invalid results

System Architecture & Bottlenecks

Typical Web Architecture

Client → Load Balancer → Web Server → App Server → Cache
→ External APIs → Database
→ Cache
→ ...



Common Bottlenecks

- Database locks
- Missing indexes
- Thread pool exhaustion
- Connection pool limits
- Garbage collection
- Blocking I/O
- External API latency

Load Testing Tools

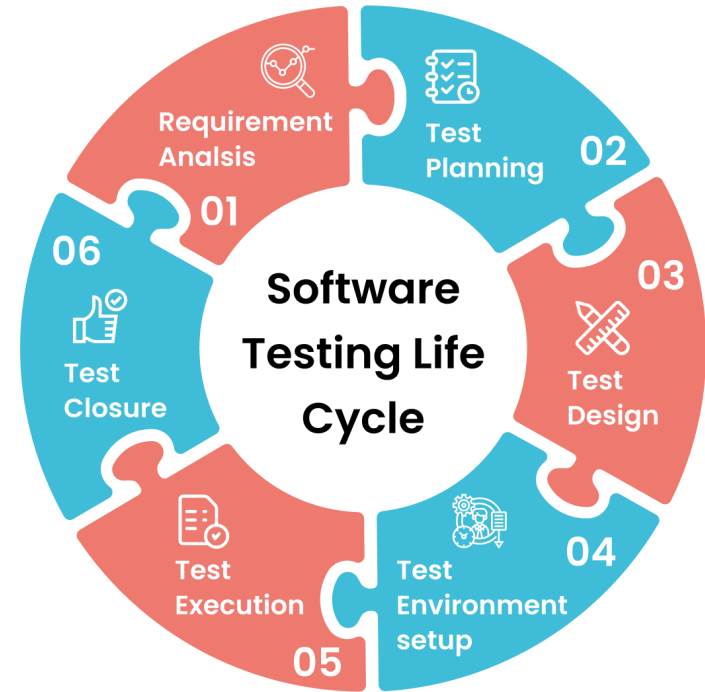
Tool	GUI	Scripting	CI/CD Friendly	Cloud
Apache JMeter	Yes	Optional	Medium	No
k6 (Studio)	Limited	Yes	High	Yes
Gatling	Limited	Yes	High	Yes
Locust	Limited	Yes	High	Yes



Test Execution Strategy

Load Test Lifecycle - Components

- Define goals (SLOs, SLAs)*
 - SLO: Metric, Goal, Timeframe
 - SLA: Contract
- Establish baseline
- Design scenarios
- Execute test
- Monitor system
- Analyze results
- Optimize & retest



www.optimumbrew.com

*<https://www.atlassian.com/de/incident-management/kpis/sla-vs-slo-vs-sli>

Test Environment Considerations

- Production-like?
- Same scaling?
- Same data volume?
- Same infrastructure limits?
- Same configuration?

Results Analysis

Interpreting Results

- Look for:
 - Increasing response time
 - Throughput plateau
 - Error rate spikes
 - Resource saturation
 - Detect Backpressure

Detecting the Saturation Point

- System behavior typically:
 - Linear scaling
 - Knee point
 - Queue growth
 - Error increase
 - Collapse

Scaling & Optimization

Scaling Strategies

- Vertical Scaling (Scale Up)
- Horizontal Scaling (Scale Out)
- Auto-Scaling (Cloud)

- Trade-offs:
 - Cost
 - Complexity
 - Latency

Optimization Techniques

- Caching
- Database indexing
- Query optimization
- Async processing
- Load balancing
- CDN usage
- Connection pool tuning

Load Testing in CI/CD

Continuous Performance Testing

- Performance budgets
- Automated regression detection
- Threshold-based pipeline failures
- Canary deployments
- Example workflow:
 - Commit → Build → Deploy to staging → Run Testsuite → Evaluate

Common Pitfalls

Typical Mistakes

- Testing on localhost
- No monitoring
- Ignoring percentiles
- No warm-up phase
- Unrealistic data
- Only testing happy path

Additions

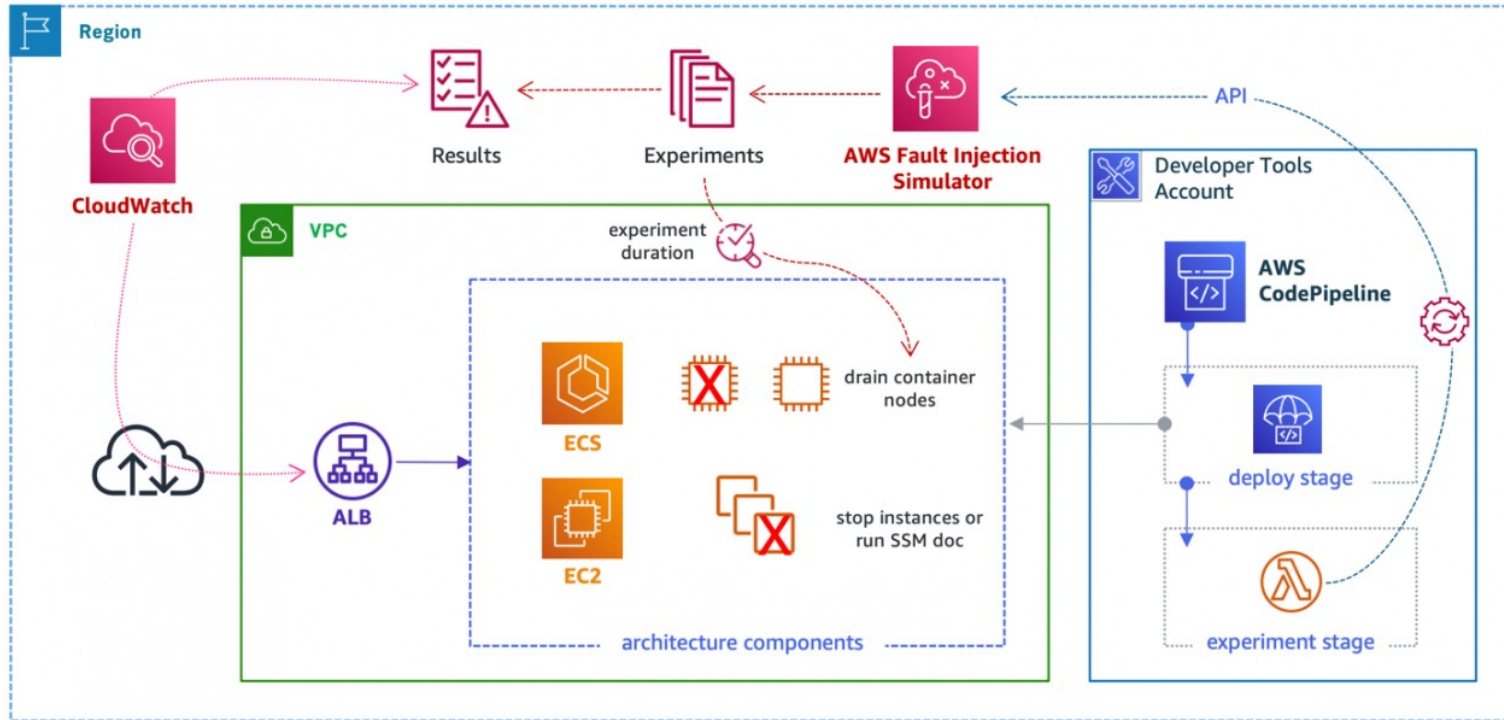
Chaos Monkey Testing

- What is Chaos Monkey?
 - Chaos Monkey is a tool originally developed by Netflix
 - It randomly terminates instances in production
 - Goal: ensure systems are resilient to failures
- Chaos Engineering Principles
 - Systems should fail gracefully
 - Failures are inevitable in distributed systems
 - Test resilience before real incidents happen
 - Build confidence in auto-recovery mechanisms

Chaos Monkey Testing – Scenarios



Chaos Monkey Testing – Examples



Real User Monitoring (RUM) vs. Synthetic Testing

RUM	Synthetic Testing
Collects performance data from real users in production	Uses scripts to simulate user behavior
Runs in the browser (e.g., JavaScript snippets)	Runs in controlled environments
Captures: • Page load times • User interactions • Device & network conditions	Can be scheduled (e.g., every minute)
👉 Reflects actual user experience	👉 Provides repeatable and comparable results

Performance Anti Patterns

- n+1 Query Problem
 - Repeated database calls instead of batching
- Missing Caching
 - Recomputing or refetching identical data
- Synchronous Blocking Calls
 - Threads waiting on I/O or external services
- Unbounded Concurrency
 - Too many threads → resource exhaustion
- Chatty APIs
 - Many small requests instead of fewer aggregated ones

Exercise

Exercise

- Scenario:
 - “Your REST API should handle 1,000 concurrent users with $p95 < 500$ ms.”
- Tasks:
 - Design workload model
 - Implement test in k6/Gatling/jMeter/...
 - Execute test
 - Identify bottleneck
 - Propose optimization